SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	K.Deepika	**Reg. No.:**	192371042
Section / Batch:		**Date:**	16-07-2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:	**100 Marks**	

SECTION A – APPLICATION BASED QUESTIONS

Q1. ***Search Engine Optimization System***

A search engine processes queries using the recurrence T(n) = T(n/2) + 1. Apply the substitution method to expand the recurrence step-by-step. Derive the final time complexity and simplify the result. Analyze how the recursion depth influences the performance of the system. Interpret the result using asymptotic notation and explain why this approach is efficient for large datasets

1. Search Engine Optimization System

Problem

A search engine processes queries using the recurrence:

T(n)=T(n/2)+1T(n)=T(n/2)+1T(n)=T(n/2)+1

Use the substitution method to solve the recurrence, derive the time complexity, and explain its efficiency.

Solution using the Substitution Method

Given:

T(n)=T(n/2)+1T(n)=T(n/2)+1T(n)=T(n/2)+1

Step 1: Expand the recurrence

T(n)=T(n/2)+1T(n)=T(n/2)+1T(n)=T(n/2)+1

Substitute for T(n/2)T(n/2)T(n/2):

T(n)=[T(n/4)+1]+1T(n)=\left[T(n/4)+1\right]+1T(n)=[T(n/4)+1]+1 =T(n/4)+2= T(n/4)+2=T(n/4)+2

Expand again:

=T(n/8)+3= T(n/8)+3=T(n/8)+3

Expand further:

=T(n/16)+4= T(n/16)+4=T(n/16)+4

After **k** expansions,

T(n)=T(n2k)+kT(n)=T\left(\frac{n}{2^k}\right)+kT(n)=T(2kn)+k

Step 2: Find the stopping condition

Recursion stops when

n2k=1\frac{n}{2^k}=12kn=1

Therefore,

2k=n2^k=n2k=n

Taking logarithm,

k=log2nk=\log_2 nk=log2n

Step 3: Substitute back

T(n)=T(1)+log2nT(n)=T(1)+\log_2 nT(n)=T(1)+log2n

Since T(1)=cT(1)=cT(1)=c (constant),

T(n)=c+log2nT(n)=c+\log_2 nT(n)=c+log2n

Ignoring constants,

T(n)=O(logn)\boxed{T(n)=O(\log n)}T(n)=O(logn)

Recursion Depth Analysis

- At every recursive call, the input size becomes **half**.
- Hence, only $\log_2 n$ recursive levels are required.
- A smaller recursion depth reduces function calls, execution time, and stack usage.
- Even if the dataset doubles, only **one extra recursive level** is added.

Asymptotic Interpretation

Time Complexity

O(logn)\boxed{O(\log n)}O(logn)

Space Complexity

Recursive stack depth:

O(logn)\boxed{O(\log n)}O(logn)

Performance Interpretation

- Query processing remains efficient even for millions of records.
- Data reduction by half minimizes unnecessary computations.
- Suitable for search engines, binary search, indexed databases, and decision trees.
- Ensures fast response time and better scalability.

Conclusion

The recurrence solves to **O(log n)** because each recursive step halves the search space. The logarithmic growth makes the algorithm highly scalable and ideal for large search engine datasets.

Q2. ***Rank Comparison System***
A university ranking system compares each student's performance with every other student to determine overall rankings using a pairwise comparison approach. Explain how nested comparisons influence the time complexity of the algorithm and describe the growth rate as the number of students increases. Determine the asymptotic time complexity using Big-O notation and justify your reasoning. Additionally, analyze the space complexity of the system, considering storage of intermediate comparisons. Interpret how this approach impacts performance for large datasets and discuss its efficiency.

2. Rank Comparison System

Problem

A university ranking system compares every student's performance with every other student using pairwise comparisons.

Working Principle

For n students:

- Student 1 compares with all remaining students.
- Student 2 compares with remaining students.
- This continues until every pair has been compared.

This results in nested comparisons.

Time Complexity Analysis

Total comparisons:

(n−1)+(n−2)+(n−3)+⋯+1(n-1)+(n-2)+(n-3)+\cdots+1(n−1)+(n−2)+(n−3)+⋯+1

Using the arithmetic series formula,

n(n−1)2\frac{n(n-1)}{2}2n(n−1)

Expanding,

=n2−n2=\frac{n^2-n}{2}=2n2−n

Ignoring constants and lower-order terms,

O(n2)\boxed{O(n^2)}O(n2)

Growth Rate

Numberof Students	Comparisons
10	45
100	4,950
1,000	499,500
10,000	49,995,000

The number of comparisons grows quadratically, meaning execution time increases rapidly as the number of students increases.

Space Complexity Analysis

Case 1: Comparisons processed immediately

If only current comparison results are stored:

O(1)\boxed{O(1)}O(1)

Case 2: All pairwise comparison results stored

Storage required:

n(n−1)2\frac{n(n-1)}{2}2n(n−1)

Therefore,

O(n2)\boxed{O(n^2)}O(n2)

Performance Interpretation

Advantages

- Produces highly accurate rankings.
- Every student is evaluated fairly against all others.
- Suitable for small datasets where precision is important.

Limitations

- Comparisons increase dramatically as the number of students grows.
- Higher execution time and memory usage for large datasets.
- Not suitable for real-time ranking of very large student populations.

Asymptotic Complexity

Time Complexity

O(n2)\boxed{O(n^2)}O(n2)

Space Complexity

- Without storing comparisons:

O(1)\boxed{O(1)}O(1)

- With storing all comparison results:

O(n2)\boxed{O(n^2)}O(n2)

Conclusion

The pairwise comparison algorithm has quadratic time complexity ($O(n^2)$) because every student is compared with every other student. While it provides accurate rankings, its performance decreases significantly for large datasets due to the rapidly increasing number of comparisons and potential storage requirements.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						

Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____